

CHANDPUR SCIENCE AND TECHNOLOGY UNIVERSITY

Department of Computer Science and Engineering

B.Sc. Engineering 1st Year 2nd Semester Final Examination, 2023

Course Code: CSE 1201

Course Title: Object Oriented Programming

Time: 03 Hours

Full Marks: 210

-
- (i) Figures in the right margin indicate full marks allotted to each question.
 - (ii) Symbols and abbreviations bear their standard meaning.
 - (iii) Use separate answer script for each SECTION.
-

SECTION – A (Marks: 105)

(Answer **any three (03)** questions from this section in **Script A**)

- | Q. No. | Marks |
|---|-------|
| 1. a) Define Class. Explain the difference between a class and an object with an example. | [10] |
| b) A company wants to store employee details like Name, ID, and Salary. Implement a class <i>Employee</i> with appropriate attributes and methods. | [10] |
| c) Use an array of Employee objects for the Employee class stated in question 1(b) to store and display details of multiple employees. | [15] |
| 2. a) Define copy constructor. | [05] |
| b) Differentiate between a constructor and a destructor. | [10] |
| c) A school wants to store student information, including name, age and CGPA. Every time a student object is created, these details should be initialized using a constructor. Now implement a parameterized constructor to initialize the attributes and a member function <i>display()</i> to show student details. | [10] |
| d) Complete the following code segment: | [10] |
| <pre>class Book {
private:
 string title;
 string author;
 float price;
public:
 void display() {
 // Print book details
 }
};

int main() {
 Book book1("C++ Programming", "Bjarne Stroustrup", 140.50);
 Book book2 = book1;
 book1.display();
 book2.display();

 return 0;
}</pre> | |
| 3. a) Define Inheritance. Differentiate public, protected, and private inheritance. | [10] |
| b) Explain how inheritance helps in code reusability. | [10] |

- c) A company has two types of employees: *SalariedEmployee* and *ContractEmployee*. [15]
Both inherit from a common base class *Employee*. The system should:
- Have a base class *Employee* with a function *showDetails()*.
 - Create two derived classes *SalariedEmployee* and *ContractEmployee*, both inheriting from *Employee*.
 - A third class *Manager* inherits from both *SalariedEmployee* and *ContractEmployee*.
 - Due to multiple inheritance, *Manager* has two copies of *showDetails*.
- Now your task is to write a code segment to resolve the above problem.

4. a) What is STL? Briefly describe the components of STL. [10]
b) Why overloading of operator '<<' must return by reference? Is it possible to overload '<<' operator using member function? Explain with examples. [13]
c) Discuss the amount of memory allocated by *ob1* and *ob2* by considering the following code segment. [12]

```
class A{
public:
    int i, j;
private:
    int k;
public:
    void set(int i)
    {
        //...
    }
};

int main()
{
    A ob1, ob2;
}
```

SECTION – B (Marks: 105)

(Answer any three (03) questions from this section in Script B)

- | Q. No. | Marks |
|--|-------|
| 5. a) Define pure virtual function. How does it make a class abstract? | [10] |
| b) Differentiate between function overloading and function overriding. | [10] |
| c) A Library Management System needs a feature to search for books in multiple ways. The system should allow users to: | [15] |
| <ul style="list-style-type: none"> — Search for a book by title. — Search for a book by author name. — Search for a book by ISBN number. | |
| Implement a class <i>Library</i> that includes overloaded versions of the <i>searchBook()</i> function: | |
| <ul style="list-style-type: none"> — <i>searchBook(string title);</i> — <i>searchBook(string author, bool byAuthor);</i> — <i>searchBook(int ISBN);</i> | |

6. a) Explain dynamic cast using an example base class and derived class. [10]

b) Explain the output of the following program.

[10]

```
#include<iostream>
using namespace std;
class A{
public:
    A(int a){
        cout<<"Base A\n";
    }
};
class B{
public:
    B(int a){
        cout<<"Base B\n";
    }
};
class C: protected A, private B{
public:
    C(int a) :B(a),A(a) {
        cout<<"Derived C\n";
    }
};
int main ( ) {
    C c(10);
    B a(5) ;
}
```

c) Explain the compile time polymorphism and run time polymorphism in details with examples. [15]

7. a) Define Friend function. [05]

b) Suppose the students of CSTU, CSE are tasked to design a Library Management System for the CSTU Central Library which should allow managing information about books and members. The system has the following properties: [15]

- Each Book and Member has specific attributes and behaviors.
- The system should implement a friend function to allow a specific function (e.g., *showDetails*) to access private members of both the *Book* and *Member* classes
- The system should also maintain an array of books in the library.

Now your task is to write a code segment for the above scenario to develop the library management system.

c) A complex number is represented as: [15]

$$\text{Complex Number} = \text{real part} + i(\text{imaginary part})$$

For example, adding $(3 + 4i)$ and $(1 + 2i)$ should give $(4 + 6i)$.

Now answer the following questions based on the above scenario:

- (i) Overload the $+$ operator to add two Complex numbers and return the result.
- (ii) Implement a constructor to initialize Complex objects.
- (iii) Implement a display function to print complex numbers in the form " $a + bi$ ".

8. a) How can we restrict a function to throw certain exception? [10]

b) "Child's pointer can point to parent's object but parent's pointer cannot point to child's object" – Justify this statement. [10]

c) "A friend function can be a global function not related to any particular class. A friend function can also be a member of another class." – Explain this statement with appropriate examples. [15]

END